JavaScript is a programming language that makes websites interactive and dynamic.

In simple words, JavaScript makes a website respond when a user does something.

Important Features of JavaScript

A Lightweight

Meaning: JavaScript is designed to run without requiring a heavy setup for many common tasks.

JavaScript can run directly in a web browser without needing you to install a separate JavaScript runtime.

B. Cross-Platform

Meaning: JavaScript can run on different operating systems and devices.

C. Single-Threaded

Meaning: JavaScript normally executes one piece of JavaScript code at a time on its main thread.

Single-threaded does not mean JavaScript must wait for every operation to finish.

It means JavaScript executes code on its main thread one task at a time. The browser can handle certain operations separately while JavaScript continues working.

callbacks can run on the main thread when it becomes available

it distribute the task in client side (Browser) and server side (V8 Node.js)

D. Interpreted Language

Meaning: JavaScript is commonly described as an interpreted language because its code can be executed by a JavaScript engine without you manually compiling it first.

you write the code and run it. You don't need to manually convert it into machine code first.

The engine may use an interpreter to execute code and a JIT compiler to compile frequently executed code into machine code for better performance.

JIT stands for Just-In-Time. It means the engine can compile code into machine code while the program is running, helping it execute faster.

Client-Side JavaScript

Client-side JavaScript runs in the user's browser.and browser has v8 engine through executed javascript code

Server-Side JavaScript

Server-side JavaScript runs on a server instead of directly in the user's browser.
A server is a computer or program that receives requests and sends responses.

JavaScript can run on a server using technologies such as Node.j

Programming Paradigms in JavaScript

A programming paradigm is a style or way of writing code to solve a problem.

JavaScript supports different programming styles, including imperative and declarative programming.

1. Imperative Programming (How to do it)

We tell the computer step by step how to complete a task.

Example:Find even numbers from an array.

```
const numbers = [1, 2, 3, 4, 5, 6];
const result = [];
for (const num of numbers) {
        if (num % 2 === 0) {
            result.push(num);
            }

    }
```

Explanation:We loop through each number, check whether it is even, and add it to the result.

2. Declarative Programming (What to do)

we describe the result we want instead of writing every step manually.

Example: Find even numbers from the same array.

```
const numbers = [1, 2, 3, 4, 5, 6];

const result = numbers.filter(num => num % 2 === 0);

console.log(result); // [2, 4, 6]
```

Explanation: We tell `filter()` to select even numbers. It handles the iteration for us.

Remember:Imperative = Step-by-step instructions. Declarative = Describe the desired result.

Note: JavaScript supports both styles. Arrow functions and `async/await` are language features, not programming paradigms themselves.

JavaScript Variables

Variables in JavaScript are used to store data values. They can be declared in different ways depending on how the value should behave.

Variables can be declared using var, let, or const.
JavaScript is dynamically typed, so types are decided at runtime.

You don’t need to specify a data type when creating a variable.

1. var keyword

   var is a keyword in JavaScript used to declare variables and it is Function-scoped and hoisted, allowing redeclaration but can lead to unexpected bugs.

2. let keyword

   let is a keyword in JavaScript used to declare variables and it is Block-scoped and not hoisted to the top, suitable for mutable variables

3. const keyword

   const is a keyword in JavaScript used to declare variables and it is Block-scoped, immutable bindings that can't be reassigned, though objects can still be mutated.

A=> let or const are preferred over var: Initially, all the variables in JavaScript were written using the var keyword but in ES6 the keywords let and const were introduced. The main issue with var is, scoping.

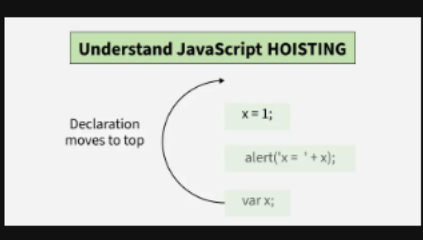
B=> var is function scoped: Can be accessed outside block if within the function.

C=> let and const are block scoped : Cannot be accessed outside block even if inside the same function

D=> var can be redeclared in the same scope, but let and const cannot be

E=> We can change elements of array or objects even if declared as const.

JavaScript Hoisting



Hoisting refers to the behavior where JavaScript moves the declarations of variables, functions, and classes to the top of their scope during the compilation phase. This can sometimes lead to surprising results, especially when using var, let, const, or function expressions.

Hoisting applies to variable and function declarations.

Initializations are not hoisted, they are only declarations.

'var' variables are hoisted with undefined, while 'let' and 'const' are hoisted but remain in the Temporal Dead Zone until initialized.

Temporal Dead Zone (TDZ)

The let and const variables are not accessible before they are initialized with some value, and the phase between the starting of the execution of block in which the let or const variable is declared till that variable is being initialized is called Temporal Dead Zone for the variable. And during this zone javascript will always through a reference error if anyone tries to access those variables

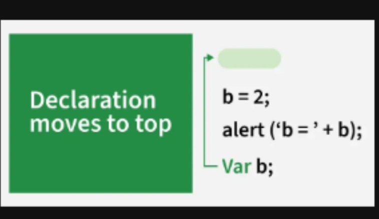
Types of Hoisting

Hoisting in JavaScript refers to moving declarations to the top of their scope before code execution.

1. Variable Hoisting with var

   When you use var to declare a variable, the declaration is hoisted to the top, but its value is not assigned until the code execution reaches the variable’s initialization. This results in the variable being assigned undefined during the hoisting phase.

   Note: The declaration var a is hoisted to the top, but a is initialized with undefined. Hence, logging results in undefined.



2. Variable Hoisting with let and const

   Unlike var, let and const are also hoisted, but they remain in a Temporal Dead Zone (TDZ) from the start of the block until their declaration is encountered. Accessing them before their declaration will throw a ReferenceError.

3. Function Declaration Hoisting

   Function declarations are hoisted with both their name and the function body. This means the function can be called before its definition in the code.means it works well

4. Function Expression Hoisting

   Function expressions are treated like variable declarations. The variable itself is hoisted, but the function expression is not assigned until the line of execution. This means calling the function before its assignment will result in an error.

```
hello(); // TypeError: hello is not a function
var hello = function() {
 console.log("Hi!");
  };
```

   Note: The variable hello is hoisted, but since it's a function expression, it’s not initialized until the line is executed.

5. Hoisting with let and const in Functions

   Variables declared with let and const inside a function are hoisted to the top of the function's scope, but they remain in the TDZ. This prevents access to them before they are initialized.

6. Hoisting with Classes

   Classes are hoisted, but they cannot be accessed before they are declared, resulting in a ReferenceError.

   Note: Although the class MyClass is hoisted, it cannot be accessed before its declaration due to the TDZ, which is why the code throws a ReferenceError.

7. Re-declaring Variables with var

   With var, you can redeclare a variable within the same scope. This is a unique behavior compared to let and const.

8. Accessing Variables Declared Later in Loops

   When using var in loops, the loop variable is hoisted to the function or global scope, which can cause unexpected behavior. If you use let, the variable is block-scoped and behaves as expected.

```
for (var i = 0; i < 3; i++) {
    setTimeout(function() {
        console.log(i); // 3, 3, 3
    }, 100);
}
```

- var: All callbacks share the same variable i. When they execute, the loop has finished and i = 3.

- let: Each iteration gets its own variable binding, so callbacks print 0, 1, 2.

- const: Cannot be reassigned, so i++ causes an error.

Remember this trick

  - var → One shared box 📦
  - let → Separate box for each iteration 📦📦📦
  - const → Cannot reassign the box's variable

9. Using Hoisted Functions with Parameters

   Functions can be hoisted with their parameters, but any parameters passed to the function are still determined by the invocation, not by the hoisting.

   Note : The entire function, including its parameters, is hoisted and available for use before the function's declaration in the code.

10. Hoisting in Nested Functions

    Hoisting works within nested functions as well. Variables declared with var inside a function are hoisted to the top of that function scope.